

SALVO Recruitment Task

Team- We Love Salvo

Programming Assignment Similarity Detection

Technical Documentation

Local, evidence based programming assignment similarity investigation

Problem Statement

Programming assignments can appear different while still retaining substantial computational and structural similarity. Variable names can be changed, formatting can be rearranged, comments can be added, and portions of the implementation can be rewritten without removing the underlying patterns being shared. At the same time, independently written solutions can also look similar when the assignment is tightly specified. The problem therefore requires more than exact text matching.

We have been building a local AI assisted investigation system for comparing programming assignments. The system is designed for identifying pairs or groups having strong measurable similarity, even when superficial source changes have been introduced. It produces similarity evidence rather than making an academic misconduct decision. The final interpretation remains with the professor or authorised reviewer.

The problem statement has therefore been reframed around evidence: What has been similar, how strongly has it been similar, which parts have been contributing to that similarity, and has the relationship been unusual within the submitted class corpus? This framing allows the system to be useful without treating a single score as proof of copying.

Existing Solutions and Their Limitations

- **Exact text matching:** It compares characters or lines directly. It has been fast, but renaming, formatting changes, comments, and small rewrites have been reducing its usefulness.
- **Token matching:** It compares programming tokens after superficial details are normalised. It has been more robust, but common assignment code and standard constructs have still been producing overlap.
- **AST comparison:** It compares syntax structure instead of raw text. It handles formatting and naming changes better, but independently produced solutions can still have similar syntax.
- **Control flow comparison:** It compares branches, loops, returns, and joins. It adds useful structural evidence, but similar control flow has not necessarily meant identical implementation.
- **Semantic models:** They have been representing code in learned vector spaces. They have been detecting deeper similarity, but a semantic score alone has been difficult to justify and has not been sufficient for a reviewer.
- **Single ML prediction:** It combines evidence automatically, but a single unexplained prediction makes review difficult, especially when training data has been limited or synthetic.

What We Have Built

We have built a local Streamlit application in which submissions have been ingested safely, parsed without execution, compared using multiple detectors, calibrated against the submitted corpus, and presented through an evidence focused interface. Python, C++, and Java have been supported, with same language comparisons being performed in the current implementation.

The system combines normalised token similarity, canonical AST similarity, lightweight CFG similarity, corpus aware common code adjustment, optional local CodeBERT semantic similarity, and a local XGBoost review priority model. Relationship groups are also derived from the pair results, allowing related submissions to be inspected as a group instead of only as isolated pairs.

Novelty of our Approach

The novelty comes from the evidence architecture rather than from claiming that one AI model has been solving plagiarism detection. Multiple independent measurements are retained, the influence of common code is reduced, corpus relative statistics are calculated, and the ML stage is placed after evidence extraction. The reviewer has therefore been able to see both the result and the measurements producing it.

The system was also designed locally. Student source does not require an external LLM or hosted plagiarism service. Code is treated as untrusted data, and submitted programs are never executed. This combination addresses privacy and security while keeping the investigation technically inspectable.

Why This Ensembled Design and Not Just a Single Detector

No single detector captures every form of similarity. Tokens capture direct reuse, AST structure captures syntax patterns, CFG structure captures control flow, and CodeBERT captures learned semantic relationships. Combining them has been reducing dependence on any one failure mode. Keeping the individual signals has also been allowinging a reviewer to understand disagreement between detectors.

End to End Flow

- **Ingesting:** Files or ZIP archives have been validated, decoded, hashed, and classified by language.
- **Analysing:** Tokens, AST structures, CFG structures, source statistics, and optional embeddings have been generated.
- **Comparing:** Every same language pair receives a common feature representation.
- **Calibrating:** Common code has been adjusted and scores have been positioned within the language specific corpus.
- **Scoring:** The deterministic evidence score has been retained, and XGBoost has been optionally producing a review priority probability.
- **Explaining:** Matched regions, shared structural blocks, feature values, and XGBoost feature contributions have been shown.

Detailed Technical Description

1. Similarity Scoring Algorithm

What: A deterministic baseline has been formed from three complementary measurements, common adjusted token similarity, AST similarity, and CFG similarity. How: When all three structural signals have been available, the baseline is calculated as 0.40 times common adjusted token similarity, plus 0.40 times AST similarity, plus 0.20 times CFG similarity. Why: The weighting has been keeping direct code overlap and syntax structure as the strongest static evidence, while control flow contributes an additional structural signal. Why this and not exact matching: exact matching has been failing under harmless source transformations. Why this and not only AST: AST similarity has been missing some direct lexical evidence. Why this and not only CFG: CFG describes flow rather than all source structure.

2. Token and Common Code Analysis

What: Normalised programming tokens are being compared, while common tokens across the corpus have been identified. How: Identifiers, literals, comments, and language syntax are represented in a controlled token form, with common corpus tokens being downweighted through a separate adjusted similarity measure. Why: Students have been receiving the same assignment, so shared boilerplate needs to contribute less evidence. Why this and not raw token overlap: raw overlap has been confusing assignment commonality with pair specific similarity.

3. AST Analysis

What: Canonical syntax signatures have been generated for Python, C++, and Java. How: Python AST and Tree sitter parsing have been used, with identifiers and literals being normalised into structural categories and structural labels being compared. Why: Renaming and formatting have been changing source text without necessarily changing program structure. Why this and not raw source comparison: canonical structure preserves more of the programming relationship while ignoring several superficial changes.

4. CFG Analysis

What: Lightweight control flow graphs have been constructed. How: Branches, loops, returns, exceptions, breaks, continues, and join points are represented as graph evidence. Why: Two programs can have different tokens while following very similar control flow. Why lightweight CFG and not full compiler analysis: the objective has been similarity investigation, not program verification, so a bounded and interpretable structural representation has been sufficient.

5. Corpus Calibration

What: Each pair receives language specific percentile and robust MAD based statistics. How: The pair score is compared with the distribution of same language pairs, with median absolute deviation being used for robust scaling. Why: A score is meaningful only in context. A highly constrained assignment can naturally produce many high similarity pairs. Why this and not a fixed threshold alone: corpus conditions have been varying between assignments and classes.

6. CodeBERT Semantic Analysis

What: Local CodeBERT adds learned semantic evidence to the static detectors. **How:** microsoft/codebert-base is loaded locally, source is tokenised with a 512 token limit, hidden states are mean pooled using the attention mask, embeddings are normalised, and cosine similarity is calculated. Embeddings are cached using a source and model hash. **Why:** semantic similarity helps when surface syntax has been changed substantially. **Why this and not an external AI API:** student source remains local, avoiding an external data transfer dependency and keeping the investigation reproducible.

7. Why XGBoost Has Been Needed for Dynamic Weightage

What: XGBoost is used as a local review priority model over the measured evidence. **How:** a fixed 44 feature contract feeds a gradient boosted tree classifier, with token, AST, CFG, semantic, size, parsing, and corpus features being considered together. **Why:** manually fixed weights have been unable to represent nonlinear interactions. For example, a moderate token similarity combined with unusually high AST similarity, CFG similarity, and semantic similarity can carry a different meaning from any one signal in isolation. XGBoost allows the weighting of evidence to be learned from labelled examples instead of remaining entirely hand tuned.

Why this and not replacing the algorithm completely: the deterministic score has been retained as `deterministic_score`, so the learned layer remains traceable to the underlying evidence. The current bundled XGBoost model was trained using synthetic demonstration labels, so its output is treated as a review priority score and not as a real world plagiarism probability. A production model requires professor reviewed relationship labels and student aware validation.

8. XGBoost Explainability

What: Pair specific model contributions have been exposed. **How:** native XGBoost prediction contributions are retrieved, providing a base log odds value and a signed contribution for each feature. **Why:** a global feature importance chart describes the model overall, but it has not been explaining a selected pair. Showing pair contributions makes the result more defensible because the reviewer can see which measured features have pushed the model output upward or downward.

Conclusion

The completed system moves beyond exact code matching by combining lexical, syntactic, control flow, semantic, corpus relative, and learned evidence. Each component answers a different part of the similarity question, while the evidence remains inspectable. The local design protects source privacy, the ingestion layer avoids program execution, and the XGBoost layer adds learned dynamic weighting without hiding the deterministic evidence underneath it.

The project has therefore been functioning as a similarity investigation system, not an automated academic misconduct judge. The strongest next validation step involving professor reviewed examples, measuring false positives and false negatives, and retraining the XGBoost layer using real relationship labels. The final decision remains with the human reviewer, supported by the evidence presented by the system.